

tail-recursive, using the techniques we discussed in the previous chapter. Here is its signature:¹¹

```
def foldLeft[A,B](as: List[A], z: B)(f: (B, A) => B): B
```

EXERCISE 3.11

Write `sum`, `product`, and a function to compute the length of a list using `foldLeft`.

EXERCISE 3.12

Write a function that returns the reverse of a list (given `List(1,2,3)` it returns `List(3,2,1)`). See if you can write it using a fold.

EXERCISE 3.13

Hard: Can you write `foldLeft` in terms of `foldRight`? How about the other way around? Implementing `foldRight` via `foldLeft` is useful because it lets us implement `foldRight` tail-recursively, which means it works even for large lists without overflowing the stack.

EXERCISE 3.14

Implement `append` in terms of either `foldLeft` or `foldRight`.

EXERCISE 3.15

Hard: Write a function that concatenates a list of lists into a single list. Its runtime should be linear in the total length of all lists. Try to use functions we have already defined.

3.4.1 More functions for working with lists

There are many more useful functions for working with lists. We'll cover a few more here, to get additional practice with generalizing functions and to get some basic familiarity with common patterns when processing lists. After finishing this section, you're not going to emerge with an automatic sense of when to use each of these functions.

¹¹ Again, `foldLeft` is defined as a method of `List` in the Scala standard library, and it is curried similarly for better type inference, so you can write `mylist.foldLeft(0.0)(_ + _)`.

Just get in the habit of looking for possible ways to generalize any explicit recursive functions you write to process lists. If you do this, you'll (re)discover these functions for yourself and develop an instinct for when you'd use each one.

**EXERCISE 3.16**

Write a function that transforms a list of integers by adding 1 to each element. (Reminder: this should be a pure function that returns a new `List`!)

**EXERCISE 3.17**

Write a function that turns each value in a `List[Double]` into a `String`. You can use the expression `d.toString` to convert some `d: Double` to a `String`.

**EXERCISE 3.18**

Write a function `map` that generalizes modifying each element in a list while maintaining the structure of the list. Here is its signature:¹²

```
def map[A,B](as: List[A])(f: A => B): List[B]
```

**EXERCISE 3.19**

Write a function `filter` that removes elements from a list unless they satisfy a given predicate. Use it to remove all odd numbers from a `List[Int]`.

```
def filter[A](as: List[A])(f: A => Boolean): List[A]
```

**EXERCISE 3.20**

Write a function `flatMap` that works like `map` except that the function given will return a list instead of a single result, and that list should be inserted into the final resulting list. Here is its signature:

```
def flatMap[A,B](as: List[A])(f: A => List[B]): List[B]
```

For instance, `flatMap(List(1,2,3))(i => List(i,i))` should result in `List(1,1,2,2,3,3)`.

¹² In the standard library, `map` and `flatMap` are methods of `List`.

**EXERCISE 3.21**

Use `flatMap` to implement `filter`.

**EXERCISE 3.22**

Write a function that accepts two lists and constructs a new list by adding corresponding elements. For example, `List(1, 2, 3)` and `List(4, 5, 6)` become `List(5, 7, 9)`.

**EXERCISE 3.23**

Generalize the function you just wrote so that it's not specific to integers or addition. Name your generalized function `zipWith`.

LISTS IN THE STANDARD LIBRARY

`List` exists in the Scala standard library (API documentation at <http://mng.bz/vu45>), and we'll use the standard library version in subsequent chapters. The main difference between the `List` developed here and the standard library version is that `Cons` is called `::`, which associates to the right,¹³ so `1 :: 2 :: Nil` is equal to `1 :: (2 :: Nil)`, which is equal to `List(1, 2)`. When pattern matching, `case Cons(h, t)` becomes `case h :: t`, which avoids having to nest parentheses if writing a pattern like `case h :: h2 :: t` to extract more than just the first element of the `List`.

There are a number of other useful methods on the standard library lists. You may want to try experimenting with these and other methods in the REPL after reading the API documentation. These are defined as methods on `List[A]`, rather than as stand-alone functions as we've done in this chapter:

- `def take(n: Int): List[A]`—Returns a list consisting of the first `n` elements of this
- `def takeWhile(f: A => Boolean): List[A]`—Returns a list consisting of the longest valid prefix of this whose elements all pass the predicate `f`
- `def forall(f: A => Boolean): Boolean`—Returns `true` if and only if all elements of this pass the predicate `f`
- `def exists(f: A => Boolean): Boolean`—Returns `true` if any element of this passes the predicate `f`

¹³ In Scala, all methods whose names end in `::` are right-associative. That is, the expression `x :: xs` is actually the method call `xs :: (x)`, which in turn calls the data constructor `::(x, xs)`. See the Scala language specification for more information.